Lab 17 — Hands-on lab: package, deliver, roll back

Goal: install Helm, discover that k3s already uses it, package the full stack (web, api, db) into a chart, install it, upgrade it, roll it back, render the same thing with Kustomize, then make the cluster pull from the lab 10 registry, by tag and then by digest. And, to close the series, clean everything up.

Prerequisites — Lab 16 done: k3s; `localhost/api:1.0` and `localhost/api:1.1` imported into containerd (lab 12); the `registry:2` image and `api:1.1` in Podman (labs 10 and 12).

Files provided (`files/`) - `stack/` — the Helm chart: `Chart.yaml`, `values.yaml`, `values-prod.yaml`, `templates/`, `files/`. - `kustomize/base/` and `kustomize/overlays/prod/` — the same API as a Kustomize base + overlay.

Step 1 — Install Helm, and see it already at work

```
mkdir -p ~/labo-docker/17 && cd ~/labo-docker/17 && cp -r <lab-path>/files/* .
curl -fsSL https://get.helm.sh/helm-v4.3.1-linux-amd64.tar.gz | tar xz -C /tmp
sudo install /tmp/linux-amd64/helm /usr/local/bin/helm
helm version --short
helm list -A
```

Observe `v4.3.1+gc0d08d0`, then `Error: kubernetes cluster unreachable: Get "http://localhost:8080/version": dial tcp [::1]:8080: connect: connection refused`.

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
helm list -A
kubectl get helmcharts -n kube-system
```

Observe two releases in `kube-system`: `traefik` and `traefik-crd`, chart `traefik-34.2.1+up34.2.0`, `deployed` since k3s was installed; and the `HelmChart` objects that created them.

Explanation. Helm is a plain client: without `KUBECONFIG` it does not know where the cluster is. And the Ingress controller you have used since lab 13 is a Helm chart installed by k3s.

> **→ WINDOWS / WSL**
> Add `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` to your `~/.bashrc`: Helm, but also k9s or Lens installed on the Windows side, need it. The k3s `kubectl` finds the file on its own.

Step 2 — Read the chart, render it, install it

```
ls -R stack | head -20
helm lint stack
helm template stack ./stack | grep -E '^kind:|^  name:' | paste - - | head -12
helm template stack ./stack --set api.replicas=5 | grep -m1 'replicas:'
```

Observe the chart's five directories and files; `1 chart(s) linted, 0 chart(s) failed` (an `[INFO] icon is recommended` is harmless); eleven rendered objects (`Secret db-secret` , three `ConfigMap` , three `Service` , two `Deployment` , the `StatefulSet db` , the `Ingress web`); and `replicas: 5` as soon as you override a value.

```
helm install stack ./stack -n lab17 --create-namespace
kubectl config set-context --current --namespace=lab17
sleep 25 ; kubectl get pods
helm list
curl -s localhost/api/ ; echo
```

Observe (a `SessionAffinity is ignored for headless services` warning is harmless) `STATUS: deployed` , `REVISION: 1` , then the text of `NOTES.txt` with the rendered values; four `Running` Pods (`api` ×2, `db-0` , `web`); `stack lab17 1 … deployed stack-0.1.0 1.0` ; and, through the Ingress and the nginx proxy, `{"message":"bonjour depuis Helm","profile":"dev"}` .

Explanation. One command created eleven objects from a directory and a values file. `helm template` shows them, `helm lint` checks them, `helm install` applies them and numbers the revision.

Step 3 — Upgrade, inspect, roll back

```
helm upgrade stack ./stack --reuse-values --set api.message="bonjour depuis la revision 2" --
set api.replicas=3
kubectl rollout status deployment/api --timeout=90s
curl -s localhost/api/ ; echo
helm history stack
helm get values stack
```

Observe `Release "stack" has been upgraded. Happy Helming!` ; a rolling update of the API; the new message in the response; two revisions (`1 superseded` , `2 deployed … Upgrade complete`); and under `USER-SUPPLIED VALUES` , exactly your two `--set` s.

```
helm rollback stack 1
kubectl rollout status deployment/api --timeout=90s ; sleep 5
curl -s localhost/api/ ; echo
kubectl get pods -l app=api
helm history stack
```

Observe `Rollback was a success!` ; the original message is back; two `api` Pods (no longer three); and a **third** revision `deployed` , described as `Rollback to 1` : the two previous ones are `superseded` .

Explanation. Changing a ConfigMap value redeployed the API thanks to the `checksum/config` annotation. The rollback does not erase history: it extends it with a revision identical to the first.

Step 4 — One values file per environment

```
cat stack/values-prod.yaml
helm template stack ./stack -f stack/values-prod.yaml | grep -E 'replicas:|APP_PROFILE|memory'
helm upgrade stack ./stack -f stack/values-prod.yaml --dry-run 2>&1 | grep -c 'kind:'
```

Observe a file containing **only the differences** from `values.yaml`; in the rendering, `APP_PROFILE: "prod"`, `replicas: 3` for the API, `memory: 512Mi` / `128Mi`, `replicas: 2` for web; and a `--dry-run` rendering eleven objects without touching anything.

Explanation. Templates never change from one environment to the next; only the values file passed with `-f` changes. It is the `compose.prod.yaml` of lab 09, for a cluster.

Step 5 — The same thing with Kustomize

```
cat kustomize/overlays/prod/kustomization.yaml
kubectl kustomize kustomize/overlays/prod | grep -E
'^kind:|namespace:|replicas:|image:|env:|memory:'
kubectl apply -k kustomize/overlays/prod
sleep 20 ; kubectl get pods -n lab17-prod -o custom-
columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,ENV:.metadata.labels.env
kubectl delete -k kustomize/overlays/prod
```

Observe an overlay referencing `../../base` and declaring `namespace`, `labels`, `replicas`, `images`, `patches`; a rendering where the untouched base has become a Deployment with `replicas: 3`, image `localhost/api:1.1`, label `env: prod` and patched resources, in `lab17-prod`; three matching Pods; then the deletion of the namespace and its content.

Explanation. No template: plain YAML, and an overlay saying what changes. `kubectl` can render (`kustomize`) and apply (`-k`) without installing anything.

Step 6 — Pull from the registry, by tag then by digest

On the Podman side (lab 10): restart the registry and push `api:1.1`, noting its digest.

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.1 localhost:5000/api:1.1
podman push --tls-verify=false --digestfile digest.txt localhost:5000/api:1.1 2>&1 | tail -1
cat digest.txt ; echo
podman image inspect api:1.1 --format '{{.Digest}}'
```

Observe `Writing manifest to image destination`, a digest `sha256:b82020cb…` in `digest.txt`, and a **different** digest (`sha256:e1b2ba1e…`) for the local image: the push recompressed the layers; the registry's is the only one that counts.

```
curl -s localhost:5000/v2/api/tags/list ; echo
helm upgrade stack ./stack --reuse-values --set api.image.repository=localhost:5000/api --set
api.image.tag=1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get events --field-selector reason=Pulled | grep -m1 5000 | cut -c1-150
kubectl get pods -l app=api -o jsonpath='{.items[0].status.containerStatuses[0].imageID}' ;
echo
```

Observe `{"name":"api","tags":["1.1"]}`; a rolling update; `Successfully pulled image "localhost:5000/api:1.1" in 92ms` (your values differ) — k3s pulled from the registry, over HTTP, with no `registries.yaml`; and `imageID` = `localhost:5000/api@sha256:b82020cb…`: the same digest as in `digest.txt`.

```
DIGEST=$(kubectl get pods -l app=api -o
jsonpath='{.items[0].status.containerStatuses[0].imageID}' | cut -d@ -f2) ; echo $DIGEST
helm upgrade stack ./stack --reuse-values --set api.image.digest=$DIGEST
kubectl rollout status deployment/api --timeout=90s
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
helm history stack | cut -c1-110
```

Observe a Deployment whose image is now `localhost:5000/api@sha256:b82020cb…`: no more tag, a fingerprint; and five revisions in the history.

Explanation. The `1.1` tag could be rewritten tomorrow; the digest never. CI pushes with an immutable tag and records the digest; production pins the digest.

→ WINDOWS / WSL
On WSL, Podman and k3s share the same machine: `localhost:5000` is the same registry for both, and containerd accepts plain `localhost`. For a registry on another machine or with credentials: `/etc/rancher/k3s/registries.yaml`, then `sudo systemctl restart k3s`.

Clean-up — and the end of the series

```
helm uninstall stack ; sleep 15
kubectl get all,pvc
kubectl delete namespace lab17 ; kubectl config set-context --current --namespace=default
helm list -A
podman rm -f -t 0 -v registry ; podman rmi localhost:5000/api:1.1
```

Observe after `uninstall`, no Pod, Service or Deployment left… but the PVC `data-db-0` still `Bound`: Helm does not delete what the StatefulSet created. Deleting the namespace takes it away. `helm list -A` only shows Traefik now.

To remove **all** of k3s from your WSL, once you are done (irreversible, also deletes the `local-path` volumes):

```
sudo /usr/local/bin/k3s-uninstall.sh
```

The Podman images (`api:1.0`, `api:1.1`, `web:1.0`, `postgres`, `nginx`) and the `stack_pgdata` volume of lab 09 remain yours: `podman images`, `podman volume ls`, and lab 10 to sort them out.

What you must be able to state now

- `helm` without `KUBECONFIG` looks for `localhost:8080`; with it, `helm list -A` shows the Traefik installed by k3s.
- Chart `stack/`: `lint`, `template` (eleven objects), `install` (revision 1, `NOTES.txt`), `curl localhost/api/`.
- `upgrade --set` → revision 2 and a new message; `rollback 1` → revision 3 `Rollback to 1`.
- `-f values-prod.yaml`: only the differences; `--dry-run` renders without applying.
- Kustomize: `kubectl kustomize` shows, `apply -k` applies, overlay = namespace + replicas + images + patches + labels.
- Registry: `--digestfile` = registry digest = `imageID` on the node; `image@sha256:` in the Deployment.
- `helm uninstall` keeps the PVC; `k3s-uninstall.sh` removes everything.